

```

/* Website + IMN lead intake for the Win-Win CRM Dialer
* _____
* The dialer (index.html) already calls this file for:
*   ?action=list → which properties are web/IMN leads (the 🌐 filter + alerts)
*   ?action=seen → clear the unseen count
* New:
*   ?action=imn → IMN Leads webhook. Creates the property, phones and a
*               lead row highlighted RED, tagged "IMN Leads".
*
* Env vars (Netlify → Site → Environment variables):
*   SUPABASE_URL, SUPABASE_SERVICE_ROLE_KEY, ACCESS_PASSWORD (already set for a
*   IMN_WEBHOOK_SECRET new — must match the x-webhook-secret header in IMN
*
* One-time SQL in Supabase (SQL Editor) before deploying:
*
*   create table if not exists web_leads (
*     property_id text primary key references properties(id) on delete cascade,
*     source      text not null default 'Website',
*     web_lead_at timestamptz not null default now(),
*     lead_seen   boolean not null default false,
*     raw         jsonb
*   );
*/

const crypto = require('crypto');

const json = (body, code = 200) => ({
  statusCode: code,
  headers: {
    'Content-Type': 'application/json',
    'Access-Control-Allow-Origin': '*',
    'Access-Control-Allow-Headers': 'Content-Type, X-Access-Password, X-Webhook-S
    'Access-Control-Allow-Methods': 'GET, POST, OPTIONS',
  },
  body: JSON.stringify(body),
});

// — Supabase REST (same pattern as api.js) —————
async function supa(path, method = 'GET', body, extraHeaders) {
  const base = String(process.env.SUPABASE_URL || '').replace(/\/+$/, '');
  const key = process.env.SUPABASE_SERVICE_ROLE_KEY;
  if (!base || !key) throw new Error('Supabase env vars missing');
  const r = await fetch(base + '/rest/v1' + path, {

```

```

    method,
    headers: {
      apikey: key,
      Authorization: 'Bearer ' + key,
      'Content-Type': 'application/json',
      Prefer: 'return=representation',
      ...(extraHeaders || {}),
    },
    body: body === undefined ? undefined : JSON.stringify(body),
  });
  const text = await r.text();
  let data; try { data = text ? JSON.parse(text) : null; } catch { data = text; }
  if (!r.ok) throw new Error((data && (data.message || data.error)) || `Supabase
  return data;
}

// — Same login check as api.js (owner password or a caller's own) —
async function isSignedIn(event) {
  const provided = event.headers['x-access-password'] || event.headers['X-Access-
  if (!provided) return false;
  if (process.env.ACCESS_PASSWORD && provided === process.env.ACCESS_PASSWORD) re
  const hash = crypto.createHash('sha256').update(String(provided), 'utf8').diges
  const rows = await supa(`/profiles?password_hash=eq.${encodeURIComponent(hash)}`
  return Array.isArray(rows) && rows.length > 0;
}

// — Helpers —————
function safeEqual(a, b) {
  const x = Buffer.from(String(a || '')), y = Buffer.from(String(b || ''));
  return x.length === y.length && crypto.timingSafeEqual(x, y);
}

// Look for a value under several possible names, at the top level or nested
function pick(obj, keys) {
  for (const src of [obj, obj && obj.lead, obj && obj.data, obj && obj.contact, o
    if (!src || typeof src !== 'object') continue;
    for (const k of keys) {
      const v = src[k];
      if (v !== undefined && v !== null && String(v).trim() !== '') return v;
    }
  }
  return '';
}

function toEl64(s) {
  const raw = String(s || '').trim().replace(/\\.\\d+$/, '');
  const d = raw.replace(/\\D/g, '');
  if (d.length === 10) return '+1' + d;
}

```

```

    if (d.length === 11 && d[0] === '1') return '+' + d;
    return '';
}
function fmtPhone(e164) {
    const d = e164.replace(/\D/g, '').replace(/^1/, '');
    return d.length === 10 ? `(${d.slice(0, 3)}) ${d.slice(3, 6)}-${d.slice(6)} ` :
}
const normAddr = a => String(a || '').toLowerCase().replace(/[,#]/g, ' ').replac
function lastName(owners) {
    const t = String(owners || '').split(/\s+/).filter(Boolean);
    const w = t[t.length - 1] || '';
    return w ? w.charAt(0).toUpperCase() + w.slice(1).toLowerCase() : '';
}

// — IMN intake —————
async function handleImn(event) {
    const secret = process.env.IMN_WEBHOOK_SECRET;
    if (!secret) return json({ error: 'IMN_WEBHOOK_SECRET not set' }, 500);
    const got = event.headers['x-webhook-secret'] || event.headers['X-Webhook-Secre
    if (!safeEqual(got, secret)) return json({ error: 'Unauthorized' }, 401);

    let body;
    try { body = JSON.parse(event.body || '{}'); } catch { return json({ error: 'In

    const first = pick(body, ['first_name', 'firstName']);
    const last = pick(body, ['last_name', 'lastName']);
    const owners = String(pick(body, ['name', 'full_name', 'fullName', 'owner', 'ow

    const street = pick(body, ['property_address', 'propertyAddress', 'address', 's
    const city = pick(body, ['city']);
    const state = pick(body, ['state']);
    const zip = pick(body, ['zip', 'zipcode', 'zip_code', 'postal_code']);
    const address = String(street).includes(',')
        ? String(street).trim()
        : [street, city, [state, zip].filter(Boolean).join(' ')].filter(Boolean).join

    const email = String(pick(body, ['email', 'email_address']) || '');
    const rawPhones = [].concat(pick(body, ['phones']) || [],
        pick(body, ['phone', 'phone_number', 'phoneNumber', 'mobile', 'cell']) || [])
    const phones = [...new Set(rawPhones.map(p => toE164(typeof p === 'object' ? (p

    if (!address && !phones.length) return json({ error: 'Lead has no address or ph

    // Anything else IMN sends (motivation, timeline, asking price...) goes into note
    const known = new Set(['first_name', 'firstName', 'last_name', 'lastName', 'name', '
        'property_address', 'propertyAddress', 'address', 'street', 'street_address', 'cit

```

```

    'postal_code', 'email', 'email_address', 'phones', 'phone', 'phone_number', 'phoneN
const extras = Object.entries(body)
    .filter(([k, v]) => !known.has(k) && v !== null && v !== '' && typeof v !== '
    .map(([k, v]) => `${k.replace(/_/g, ' ')}: ${v}`);
const notes = ['📞 IMN Leads - call ASAP', email && `Email: ${email}`, ...extra

// Reuse the property if the address is already in the dialer
let propertyId = null;
if (address) {
    const key = normAddr(address);
    const all = await supa('/properties?select=id,property_address&limit=20000');
    const hit = (all || []).find(p => normAddr(p.property_address) === key);
    if (hit) propertyId = hit.id;
}
const isNew = !propertyId;

if (isNew) {
    propertyId = 'imn-' + Date.now() + '-' + crypto.randomBytes(3).toString('hex')
    await supa('/properties', 'POST', [{
        id: propertyId,
        owners,
        owner_last_name: lastName(owners),
        property_address: address,
        mailing_address: '',
        email,
        list_name: 'IMN Leads',
    }], { Prefer: 'return=minimal' });
}

// Add any phone numbers the property doesn't already have
if (phones.length) {
    const existing = await supa(`/phones?property_id=eq.${encodeURIComponent(prop
    const have = new Set((existing || []).map(p => p.e164));
    const rows = phones.filter(e => !have.has(e))
        .map(e => ({ property_id: propertyId, e164: e, display: fmtPhone(e), type:
    if (rows.length) await supa('/phones', 'POST', rows, { Prefer: 'return=minima
}

// Red highlight (the dialer's own colour system - clear it with ✕ after callin
// (existing leads keep their notes - only brand-new ones get IMN's details)
const leadRow = {
    property_id: propertyId,
    highlight: 'red',
    called: false,
    updated_at: new Date().toISOString(),
};

```

```

if (isNew) leadRow.va_notes = notes;
await supa('/leads?on_conflict=property_id', 'POST', [leadRow],
  { Prefer: 'resolution=merge-duplicates,return=minimal' });

// Flag as an IMN lead → 🔥 IMN · CALL ASAP tag, alert and notification
await supa('/web_leads?on_conflict=property_id', 'POST', [{
  property_id: propertyId,
  source: 'IMN',
  web_lead_at: new Date().toISOString(),
  lead_seen: false,
  raw: body,
}], { Prefer: 'resolution=merge-duplicates,return=minimal' });

return json({ ok: true, propertyId, created: isNew });
}

// — Main —————
exports.handler = async (event) => {
  if (event.httpMethod === 'OPTIONS') return json({}, 204);
  const action = (event.queryStringParameters || {}).action || '';

  try {
    if (action === 'imn') {
      if (event.httpMethod !== 'POST') return json({ error: 'POST only' }, 405);
      return await handleImn(event);
    }

    if (!(await isSignedIn(event))) return json({ error: 'Unauthorized' }, 401);

    if (action === 'list') {
      const leads = await supa('/web_leads?select=property_id,source,web_lead_at',
        return json({ leads: leads || [] });
    }
    if (action === 'seen') {
      await supa('/web_leads?lead_seen=is.false', 'PATCH', { lead_seen: true }, {
        return json({ ok: true });
      }
      return json({ error: 'Unknown action: ' + action }, 404);
    } catch (e) {
      console.error('web-lead error:', e.message);
      return json({ error: e.message }, 500);
    }
  };
};

```